

90 DAYS CHALLENGE

DAY 04 — PYTHON OPERATORS

Topic: Operators in Python — Definition, Types, Syntax & Examples

What is an Operator?

An operator is a special symbol or keyword in Python that tells the computer to perform a specific operation on one or more values. The values or variables on which an operator works are called **operands**. For example, in **a + b**, '+' is the operator and 'a' and 'b' are operands.

1. Arithmetic Operators

Operator	Definition	Syntax	Example / Explanation
Addition (+)	The addition operator is used to add two numbers or values together.	<code>a + b</code>	If <code>a = 10</code> and <code>b = 5</code> , then <code>10 + 5 = 15</code> .
Subtraction (-)	The subtraction operator subtracts the right-hand value from the left-hand value.	<code>a - b</code>	<code>10 - 5 = 5</code> .
Multiplication (*)	The multiplication operator multiplies two values. It can also repeat a string or sequence.	<code>a * b</code>	<code>10 * 5 = 50</code> .
Division (/)	The division operator divides one number by another and normally returns the result as a floating-point number.	<code>a / b</code>	<code>10 / 4 = 2.5</code> .
Floor Division (//)	The floor division operator divides two numbers and returns the floor value of the result.	<code>a // b</code>	<code>10 // 4 = 2</code> .
Modulus (%)	The modulus operator returns the remainder left after division.	<code>a % b</code>	<code>10 % 4 = 2</code> .
Exponentiation (**)	The exponentiation operator raises the left value to the power of the right value.	<code>a ** b</code>	<code>2 ** 3 = 8</code> .

2. Comparison / Relational Operators

Operator	Definition	Syntax	Example / Explanation
Equal to (==)	Checks whether two values are equal. The result is either True or False.	<code>a == b</code>	<code>10 == 10</code> gives True.
Not equal (!=)	Checks whether two values are different.	<code>a != b</code>	<code>10 != 5</code> gives True.
Greater than (>)	Checks whether the left value is greater than the right value.	<code>a > b</code>	<code>10 > 5</code> gives True.
Less than (<)	Checks whether the left value is smaller than the right value.	<code>a < b</code>	<code>5 < 10</code> gives True.
Greater than or equal (>=)	Checks whether the left value is greater than or equal to the right value.	<code>a >= b</code>	<code>10 >= 10</code> gives True.

Operator	Definition	Syntax	Example / Explanation
Less than or equal (<=)	Checks whether the left value is less than or equal to the right value.	a <= b	5 <= 10 gives True.

3. Assignment Operators

Operator	Definition	Syntax	Example / Explanation
Assignment (=)	Assigns a value to a variable. It stores the value on the right in the variable on the left.	x = 10	x now stores 10.
Add and assign (+=)	Adds a value to a variable and stores the updated result in the same variable.	x += 5	Same as x = x + 5.
Subtract and assign (-=)	Subtracts a value from a variable and stores the result.	x -= 5	Same as x = x - 5.
Multiply and assign (*=)	Multiplies a variable by a value and stores the result.	x *= 5	Same as x = x * 5.
Divide and assign (/=)	Divides a variable by a value and stores the result.	x /= 5	Same as x = x / 5.
Floor divide and assign (//=)	Performs floor division and assigns the result back to the variable.	x //= 5	Same as x = x // 5.
Modulus and assign (%=)	Finds the remainder and assigns it back to the variable.	x %= 5	Same as x = x % 5.
Exponent and assign (**=)	Raises a variable to a power and stores the result.	x **= 2	Same as x = x ** 2.

4. Logical Operators

Operator	Definition	Syntax	Example / Explanation
and	Returns True only when both conditions are True. It is used to combine multiple conditions.	a > 5 and b < 10	Both conditions must be True.
or	Returns True when at least one of the given conditions is True.	a > 5 or b < 10	One or both conditions can be True.
not	Reverses the Boolean result of a condition: True becomes False and False becomes True.	not (a > 5)	If a > 5 is True, not makes it False.

5. Bitwise Operators

Operator	Definition	Syntax	Example / Explanation
AND (&)	Compares the binary bits of two integers. A bit becomes 1 only when both corresponding bits are 1.	a & b	Performs bit-by-bit AND.
OR ()	Compares binary bits. A bit becomes 1 when at least one corresponding bit is 1.	a b	Performs bit-by-bit OR.

Operator	Definition	Syntax	Example / Explanation
XOR (^)	Compares binary bits. A bit becomes 1 when the corresponding bits are different.	<code>a ^ b</code>	Performs exclusive OR.
NOT (~)	Inverts the bits of an integer according to Python's integer representation.	<code>~a</code>	Flips the bits.
Left shift (<<)	Moves the binary bits to the left by the specified number of positions.	<code>a << 1</code>	For positive integers, one left shift generally doubles the value.
Right shift (>>)	Moves the binary bits to the right by the specified number of positions.	<code>a >> 1</code>	For positive integers, one right shift generally halves the value.

6. Membership Operators

Operator	Definition	Syntax	Example / Explanation
in	Checks whether a value is present inside a sequence or collection such as a list, tuple, string, set, or dictionary.	<code>x in items</code>	Returns True if x is found.
not in	Checks whether a value is absent from a sequence or collection.	<code>x not in items</code>	Returns True if x is not found.

7. Identity Operators

Operator	Definition	Syntax	Example / Explanation
is	Checks whether two variables refer to the same object in memory. It checks object identity, not simply equal values.	<code>a is b</code>	True when both references point to the same object.
is not	Checks whether two variables do not refer to the same object.	<code>a is not b</code>	True when they refer to different objects.

8. Conditional (Ternary) Expression

Operator	Definition	Syntax	Example / Explanation
Ternary / Conditional expression	A short form of if-else used to select one of two values based on a condition.	<code>result = 'Pass' if marks >= 40 else 'Fail'</code>	If marks are 40 or more, result is 'Pass'; otherwise it is 'Fail'.

DAY 04 — QUICK REVISION NOTES

Arithmetic:	+, -, *, /, //, %, **
Comparison:	==, !=, >, <, >=, <=
Assignment:	=, +=, -=, *=, /=, //=, %=, **=
Logical:	and, or, not
Bitwise:	&, , ^, ~, <<, >>
Membership:	in, not in
Identity:	is, is not

Important Difference: `==` compares the values of two objects, while `is` checks whether two references point to the same object. For example, two separate lists can contain the same values but still not be the same object.

Memory Tip: Arithmetic = calculation • Comparison = comparison • Assignment = storing/updating • Logical = combining conditions • Bitwise = working with bits • Membership = checking presence • Identity = checking the same object.